INDIAN INSTITUTE OF TECHNOLOGY JODHPUR · DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Anti-Proxy Classroom Attendance System Using Dynamic QR, Biometric Passkeys, and Live Location

Design Credit Project Report · Eliminating Proxy Attendance via 4-Second Rotating HMAC-SHA256 QR Codes, Hardware-Bound W3C WebAuthn Passkeys, and Live GPS Geofencing

Project Overview: Classroom attendance systems in large university courses frequently suffer from proxy attendance—where absent students ask peers to sign paper sheets, forward photographs of static QR codes over instant messaging apps, or log into multiple student accounts from a single phone inside the lecture hall. This Design Credit project report presents the architecture, visual workflows, mathematical models, and full-stack implementation of the **IIT Jodhpur Attendance Portal** (MayankMT1024/Attendance-portal). Developed in Semester 3 by **Mayank Tiwari (B25CM1075)** and **Shubham Bhatia (B25EC1050)** under the supervision of Prof. Pallavi Jain, the platform solves proxy attendance by combining four verification layers: institute Google OAuth 2.0 login, 4-second rotating HMAC-SHA256 QR tokens, 1-to-1 hardware-locked biometric passkeys, and live GPS location verification.

Project Attribution and Technical Summary

Attribute	Details
Project Title	Fair Classroom Attendance System with Dynamic QR, Biometric Authentication, and Live Location
Student Developers	**Mayank Tiwari** (Roll No. `B25CM1075`) and **Shubham Bhatia** (Roll No. `B25EC1050`)
Faculty Supervisor	Prof. Pallavi Jain, Associate Professor, Department of Computer Science & Engineering, IIT Jodhpur
Academic Course	Design Credit Project · Semester 3 (Academic Year 2026–2027)
Institution	Indian Institute of Technology Jodhpur, NH 62, Nagaur Road, Karwar, Jodhpur, Rajasthan 342030
GitHub Repository	https://github.com/MayankMT1024/Attendance-portal
Open-Source Stack	HTML5/CSS3/ES2022 JS, Vercel Serverless Node.js, Supabase PostgreSQL & Auth, SimpleWebAuthn v14 (W3C WebAuthn), html5-qrcode, QRCode.js, MDN Geolocation API, and Leaflet.js v1.9.4

Submitted By:
Mayank Tiwari (`B25CM1075`)
Shubham Bhatia (`B25EC1050`)

Faculty Supervisor:
Prof. Pallavi Jain
Dept. of CSE, IIT Jodhpur

Submission Date:
October 2026
IIT Jodhpur, Rajasthan

1. The Classroom Proxy Problem and Four-Layer Verification Solution

In university courses with 60 to 300 students, recording accurate attendance without wasting valuable lecture time is a persistent operational challenge. At the Indian Institute of Technology Jodhpur, students are required to maintain a minimum 75% attendance record across registered courses. Traditional paper sign-in sheets and verbal roll calls consume 10 to 15 minutes of class time and are easily bypassed when students sign or answer for absent friends. Basic digital alternatives—such as Google Forms or static QR codes—are fast, but students easily defeat them by taking a photo of the QR code and sending it to hostel groups, or by logging into a friend's account on a second browser tab inside the classroom.

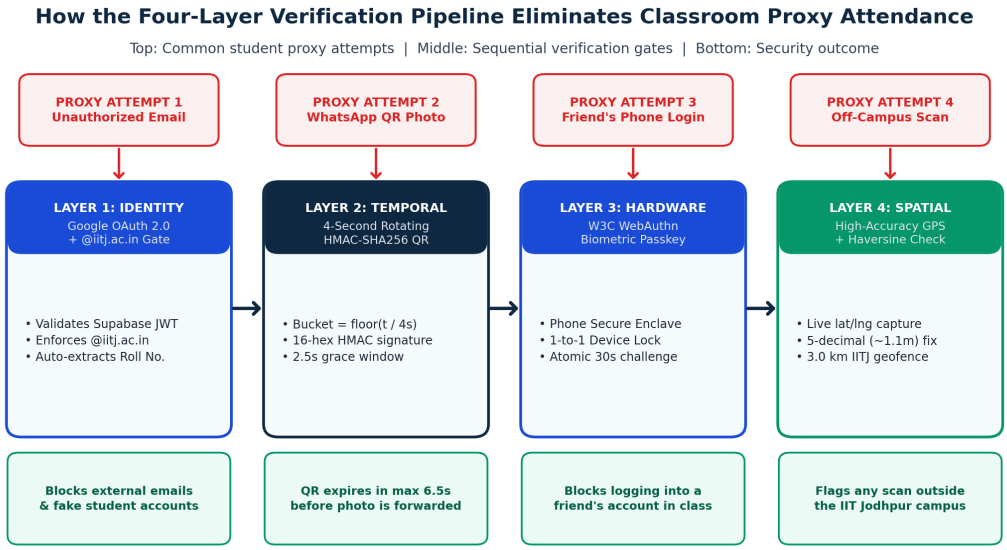


Figure 1: Visual overview of how the four sequential verification gates (Identity, Temporal HMAC QR, Hardware Biometric Passkey, and Live GPS Geofence) block every major student proxy attempt.

1.1 How Common Proxy Methods Work and How the Portal Blocks Them

As illustrated in Figure 1, the IIT Jodhpur Attendance Portal (MayankMT1024/Attendance-portal) was designed by Mayank Tiwari (`B25CM1075`) and Shubham Bhatia (`B25EC1050`) under Prof. Pallavi Jain by breaking down every method students use to mark proxy attendance and engineering a dedicated technical barrier against each one:

- **Proxy Method 1 — Sharing QR Photos on WhatsApp or Telegram:** A student inside the lecture hall takes a picture of the projected QR code and sends it to absent friends in their hostel. *How the portal solves it:* The instructor screen projects a dynamic QR code signed with RFC 2104 HMAC-SHA256 that rotates every **4 seconds** (`ROTATION_SECONDS = 4`) with a 2.5-second network grace buffer (`QR_GRACE_MS = 2500`). By the time a photo is captured, uploaded to chat, downloaded in a hostel room, and scanned, the 6.5-second validity window has already expired.
- **Proxy Method 2 — Logging into an Absent Friend's Account on One Phone:** An attending student marks their own attendance, logs out, signs into an absent friend's `@iitj.ac.in` account on the same phone, and scans the live screen again. *How the portal solves it:* Every student account is cryptographically bound to their own phone's hardware biometric sensor (Touch ID, Face ID, or fingerprint) via the W3C Web Authentication API. During check-in, the server demands a signature from that specific student's registered credential ID (`allowCredentials`), which does not exist on the friend's phone.

- **Proxy Method 3 — Temporarily Swapping Registered Phones Before Class:** A student tries to re-register their biometric passkey onto a friend's phone right before class and switch it back later. *How the portal solves it:* Once a student binds their device on first login, the server (`lib/register-options.js`) locks the binding permanently (`HTTP 409 Conflict`). Students cannot change or reset their bound device themselves; only the course instructor or Teaching Assistant (TA) can unbind a device via the Instructor Console (`/api/teacher-reset-device`).
- **Proxy Method 4 — Remote Video Streaming or Off-Campus Relay:** If a student attempts a live video call of the projector screen to someone outside the classroom, the portal's fourth gate requires live high-accuracy GPS coordinates via the MDN Geolocation API and computes the Haversine distance from the IIT Jodhpur campus center (`26.4716° N, 73.1134° E`), flagging any off-campus location on the instructor's map.

Attendance Method	Class Time Used	Stops QR Photo Sharing?	Stops Multi-Account Login on 1 Phone?	Verifies Live Campus Location?
Paper Sheet / Roll Call	10–15 minutes	No (forged signatures / proxy voice)	No (one student answers for friends)	No digital location or timestamp log
Static QR / Google Form	1–2 minutes	No (one photo works for entire class)	No (anyone with password can submit)	No location verification
Wall Biometric Scanner	10–20 minutes (door queue)	Yes (requires finger)	Yes (hardware template)	Fixed to wall, high hardware cost per room
IIT Jodhpur Attendance Portal (Our System)	**Under 30 seconds** (parallel scan)	**Yes:** 4-second HMAC-SHA256 rotating QR expires in 6.5s max	**Yes:** 1-to-1 hardware passkey lock + instructor-only device reset	**Yes:** Live GPS coordinates + Haversine 3.0 km campus map

2. Full-Stack System Architecture and Open-Source Technology Stack

The platform is engineered as a lightweight, zero-install web application divided into two role-specific interfaces: the **Student Mobile Portal** (`index.html` and `attendance.js`) and the **Instructor Console** (`teacher.html` and `teacher.js`), sharing a common helper library (`shared.js`) and stylesheet (`style.css`).

2.1 Client, Serverless API, and Database Tiers

Every operation in the portal follows a three-tier request pipeline connecting the student and instructor browsers, the Vercel Serverless Node.js API router, and the Supabase PostgreSQL backend:

1. **Tier 1 — Client Browser Applications:** Students open `index.html` on their smartphones, while instructors and TAs open `teacher.html` on the classroom podium computer. Both apps sign in via Google OAuth 2.0 pre-filtered to the `iitj.ac.in` hosted domain (`hd: 'iitj.ac.in'` in `shared.js`) and attach the resulting session token (`Authorization: Bearer <access_token>`) to every API request via `authedFetch()`.
2. **Tier 2 — Unified Serverless API Router (`api/[...route].js`):** All `/api/*` calls are handled by a single catch-all Node.js router that sets `Cache-Control: no-store` and dispatches requests to 18 specialized modules in the `lib/` directory. Consolidating routing into `api/[...route].js` keeps the deployment fast, modular, and within serverless function limits.

3. **Tier 3 — Supabase Authentication and PostgreSQL Database:** Every backend handler first invokes `authenticateUser()` in `lib/auth-guard.js`, which verifies the JWT with Supabase Auth, confirms `user.app_metadata.provider === 'google'`, and enforces that the email ends with `@iitj.ac.in` before querying the PostgreSQL tables.

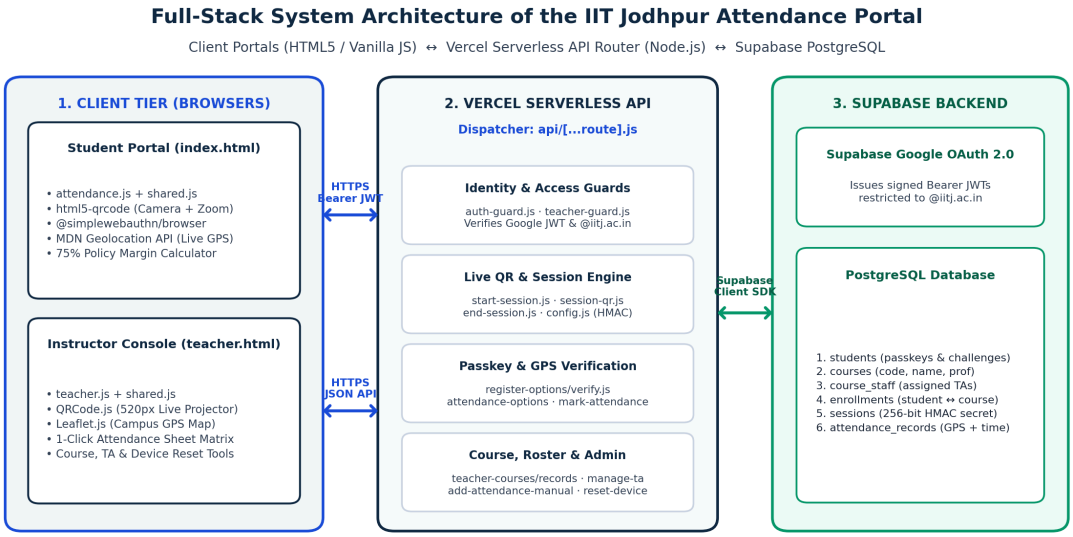


Figure 2: Three-tier architecture showing the Student Portal and Instructor Console (Left), the unified Vercel Serverless API dispatcher and 18 functional modules (Center), and Supabase OAuth 2.0 + PostgreSQL storage (Right).

Open-Source Library / API	Version / Standard	Where Used	Role in the Attendance Portal
@supabase/supabase-js	`v2.112.3`	`shared.js` & `lib/auth-guard.js`	Handles Google OAuth 2.0 sign-in, session JWT management, and server-side PostgreSQL queries
@simplewebauthn/browser & server	`v14.0.0` / `v14.0.2`	`attendance.js` & `lib/*`	Implements W3C WebAuthn hardware passkey registration and biometric challenge verification
html5-qrcode	`v2.3.8`	`index.html` & `attendance.js`	Provides 12 FPS mobile rear-camera QR scanning and hardware video track pinch-to-zoom control
QRCode.js	`v1.0.0`	`teacher.html` & `teacher.js`	Renders high-contrast `520 × 520` classroom QR codes every 4 seconds on the projector stage
MDN Geolocation API	W3C Browser Standard	`attendance.js`	Acquires live high-accuracy GPS coordinates (`enableHighAccuracy: true`) during student check-in
Leaflet.js	`v1.9.4` + OpenStreetMap	`teacher.html` & `teacher.js`	Displays interactive lecture scan maps and highlights off-campus outliers beyond 3.0 km

3. Dynamic 4-Second HMAC-SHA256 QR Rotation Protocol

To prevent students inside the classroom from photographing the projector screen and sharing the QR code with absent classmates, the QR code must change every few seconds. However, if the server had to write a new random string into the database every 4 seconds, network delays and database locks would slow down scanning for 200 students. Our system solves this using a stateless, time-bucketed cryptographic

protocol (RFC 2104 HMAC-SHA256) implemented in `lib/config.js`, `lib/start-session.js`, `lib/session-qr.js`, and `lib/attendance-options.js`.

3.1 How Time-Bucketed QR Signing Works

When an instructor starts a live session in `lib/start-session.js`, the server generates a single 256-bit secret key (`crypto.randomBytes(32).toString('hex')`) and saves it in `sessions.session_secret`. Every 4 seconds, the Instructor Console calls `GET /api/session-qr`, which computes the current 4-second time bucket $B(t)$ from the current Unix time t (in milliseconds) and signs it with HMAC-SHA256:

```
bucket = Math.floor(Date.now() / 1000 / 4);
mac = crypto.createHmac('sha256', Buffer.from(session_secret, 'hex'))
            .update(`${session_id}:${bucket}`).digest('hex').slice(0, 16);
payload = `${session_id}:${bucket}:${mac}`;
```

Taking the first 16 hex characters (64 bits of security) keeps the QR string short (~59 characters) so QRCode.js renders large, easy-to-scan squares (`CorrectLevel.L`) that phone cameras can read quickly from the back row of a lecture hall. The API also returns `ms_until_next`, allowing `teacher.js` to synchronize its refresh timer and animate a smooth countdown bar right below the QR code.

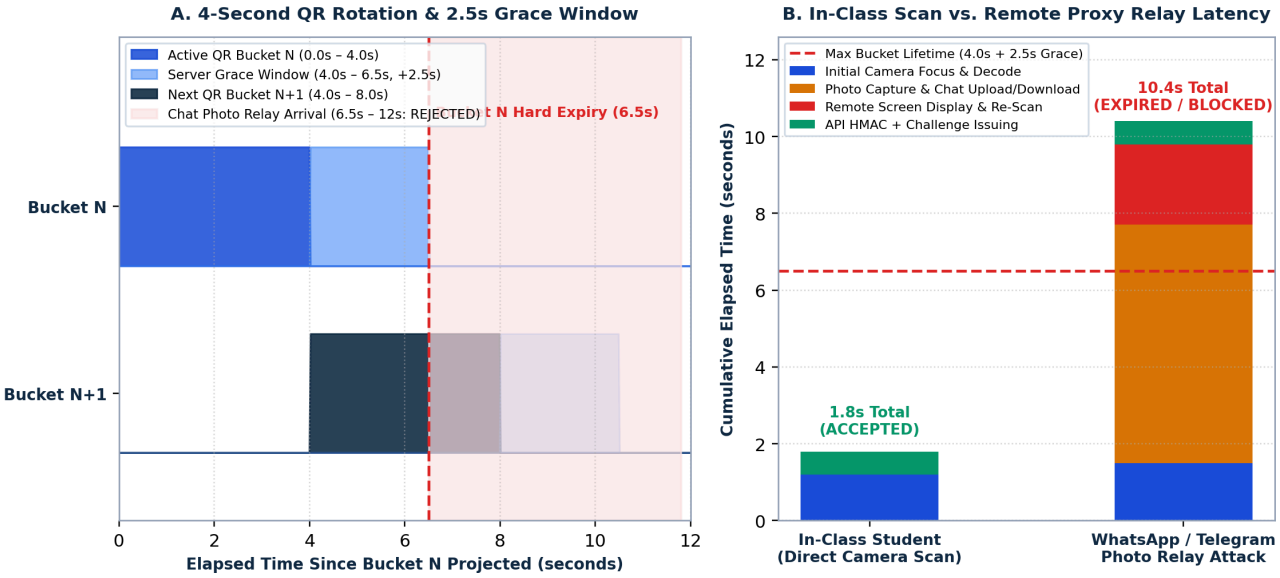


Figure 3: (A) Timeline of the 4.0-second QR bucket and 2.5-second grace overlap, and (B) end-to-end latency breakdown showing why direct in-class scans succeed in ~1.8 seconds while WhatsApp/Telegram photo relays take ~10.4 seconds and are rejected.

3.2 Why the 2.5-Second Grace Window Helps Honest Students but Blocks Proxies

If a student in class scans a QR code at 3.9 seconds—right before the projector flips to the next QR code at 4.0 seconds—their phone needs a moment to send the request over Wi-Fi or mobile data. In `lib/config.js`, the function `qrBucketIsValid(bucket, now)` accepts both the current 4-second bucket B and the immediately preceding bucket $B - 1$ for up to **2.5 seconds** (`QR_GRACE_MS = 2500`) after it leaves the screen.

As shown in Figure 3, this gives each QR code a maximum lifetime of *4.0 s + 2.5 s = 6.5 seconds*. An in-class student pointing their camera directly at the projector finishes decoding and calling `/api/attendance-options` in ~1.8 seconds. In contrast, photographing the projector, sending the image on WhatsApp/Telegram, downloading it on another device, and scanning that screen takes over 10 seconds—missing the 6.5-second window every time. Furthermore, `lib/attendance-options.js` compares the scanned HMAC and expected HMAC using `crypto.timingSafeEqual(a, b)` so attackers cannot guess the signature using timing attacks, and checks `expected_course_id` so students are alerted if they selected the wrong course.

Configuration Constant	Value	Defined In	Purpose in the Anti-Proxy Protocol
`ROTATION_SECONDS`	4 seconds	`lib/config.js`	Rotates the projected classroom QR payload every 4 seconds
`QR_GRACE_MS`	`2500` ms (2.5 s)	`lib/config.js`	Allows in-flight scans from the previous bucket to succeed while blocking chat photo forwarding
`FINGERPRINT_WINDOW_MS`	`30000` ms (30 s)	`lib/config.js`	Time window after a valid QR scan for the student to complete biometric verification and GPS lock
`SESSION_MAX_MS`	`7200000` ms (2 h)	`lib/config.js`	Auto-closes any session after 2 hours and lets teachers resume an active session if they refresh the page

4. Hardware-Bound Biometric Passkeys and Device Locking

Dynamic QR codes stop remote photo sharing, but what if a student sitting inside the classroom logs out of their own account, logs into their absent roommate's `@iitj.ac.in` account on the same phone, and scans the projector screen again? To make multi-account proxy scanning impossible, our system locks each student roll number to their personal phone's hardware biometric sensor using W3C WebAuthn and SimpleWebAuthn v14.

4.1 One-Time Passkey Binding and Instructor-Controlled Device Reset

When a student signs in for the first time, `GET /api/student-data` checks whether `students.webauthn_credential` is present. If it is `null`, the Student Portal opens the **Register this device** screen (`#screen-setup`). Tapping **Bind biometric passkey** calls `POST /api/register-options`, which configures WebAuthn with three strict security settings:

- **Built-In Phone Hardware Only (`authenticatorAttachment: 'platform'`):** Requires the passkey to be created inside the device's built-in hardware security chip (Apple Touch ID / Face ID Secure Enclave or Android Keystore fingerprint sensor). External USB keys or cross-device passkeys are blocked.
- **Mandatory Biometric Check (`userVerification: 'required'`):** The phone's hardware private key will not sign any challenge unless the live fingerprint or face unlock succeeds.
- **Strict 1-to-1 Device Lock (No Student Self-Reset):** Once `lib/register-verify.js` saves the student's credential ID and public key, `lib/register-options.js` blocks any future attempt by the student to register another device (`HTTP 409 Conflict`). If a student genuinely loses or replaces their phone, they must visit their instructor or TA, who clicks **Reset Device** in the Instructor Console (`POST /api/teacher-reset-device`) to clear `webauthn_credential` so the student can bind their new phone.

4.2 Atomic 30-Second Challenge Consumption (Stopping Replay Attacks)

Every classroom scan links the QR code and the biometric prompt into a single atomic transaction across `lib/attendance-options.js` and `lib/mark-attendance.js`:

1. When the student scans a valid QR code, `POST /api/attendance-options` verifies the QR HMAC and generates a one-time cryptographic challenge restricted to that student's registered credential ID (`allowCredentials: [{ id: student.webauthn_credential.id, type: 'public-key' }]`). It saves `current_challenge`, `pending_qr_payload = qr_payload`, `challenge_purpose = 'attendance'`, and `challenge_expires_at = now + 30s` in the `students` table.

2. When the student completes their fingerprint/Face ID prompt, `POST /api/mark-attendance` checks that the 30-second window has not expired and that `pending_qr_payload` matches the submitted `qr_payload`.
3. Before verifying the signature, `lib/mark-attendance.js` executes an atomic compare-and-clear query (`UPDATE students SET current_challenge = null ... WHERE id = student.id AND current_challenge = student.current_challenge`). This guarantees a challenge can never be replayed twice, and updates the WebAuthn signature counter (`newCounter`) after verification.

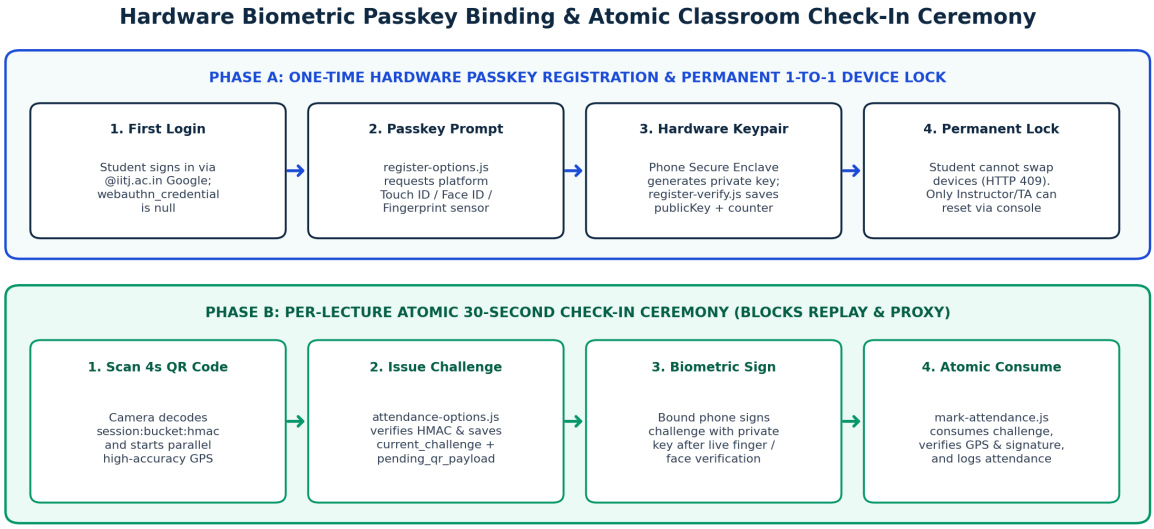


Figure 4: (Phase A) One-time hardware passkey registration and permanent 1-to-1 device lock, and (Phase B) the per-lecture 30-second atomic check-in ceremony that binds the scanned QR code to the student's biometric signature.

Security Feature	Where Implemented	How It Protects Classroom Integrity
`allowCredentials` Matching	`lib/attendance-options.js`	Ensures only the specific phone bound to that student's roll number can sign the attendance challenge
`pending_qr_payload` Lock	`lib/attendance-options.js` & `lib/mark-attendance.js`	Ties the biometric challenge to the exact QR code scanned, preventing cross-session token swapping
Atomic Challenge Clear	`lib/mark-attendance.js`	Clears `current_challenge` atomically before inserting the attendance record, blocking replay scripts
Teacher-Only Device Reset	`lib/teacher-reset-device.js`	Prevents students from swapping accounts onto a friend's phone for a single lecture

5. Live GPS Geolocation Verification, Campus Geofencing, and Permission Recovery

On top of dynamic 4-second QR codes and hardware passkeys, we added live location verification so every attendance record includes the student's physical GPS coordinates at the moment of check-in, and instructors can visually audit all scans on an interactive campus map.

5.1 Parallel High-Accuracy GPS Capture and Haversine Campus Geofencing

Getting a precise GPS lock on a phone takes 1 to 3 seconds. To keep check-in fast, `attendance.js` starts `getLocationAsync()` using `navigator.geolocation.getCurrentPosition(..., { enableHighAccuracy: true, timeout: 10000, maximumAge: 60000 })` at the exact same moment the biometric fingerprint prompt opens. By the time the student touches the fingerprint sensor, the GPS coordinates are already ready.

In `lib/mark-attendance.js`, the server validates that `latitude` $\in [-90, 90]$ and `longitude` $\in [-180, 180]$ are finite numbers and rounds them to 5 decimal places (~1.1 meters accuracy). If location is missing, attendance is blocked. On the Instructor Console (`teacher.js`), the **Location Map** tab loads all session coordinates via `GET /api/course-locations` and computes each scan's distance d from the IIT Jodhpur campus center (`CAMPUS_CENTER = [26.4716, 73.1134]`) using the Haversine formula:

$$a = \sin^2(\Delta\varphi / 2) + \cos(\varphi_0) \cdot \cos(\varphi) \cdot \sin^2(\Delta\lambda / 2)$$
$$d = 2 \cdot R \cdot \operatorname{atan2}(\sqrt{a}, \sqrt{1-a}) \quad \text{where } R = 6{,}371{,}000 \text{ meters}$$

Scans inside the 3.0 km campus boundary appear as blue markers (`#3b82f6`) on the Leaflet.js map, while any scan beyond 3.0 km (`dist > 3000`) is highlighted in amber (`#f59e0b`) with a popup showing the student's roll number and exact distance outside campus.

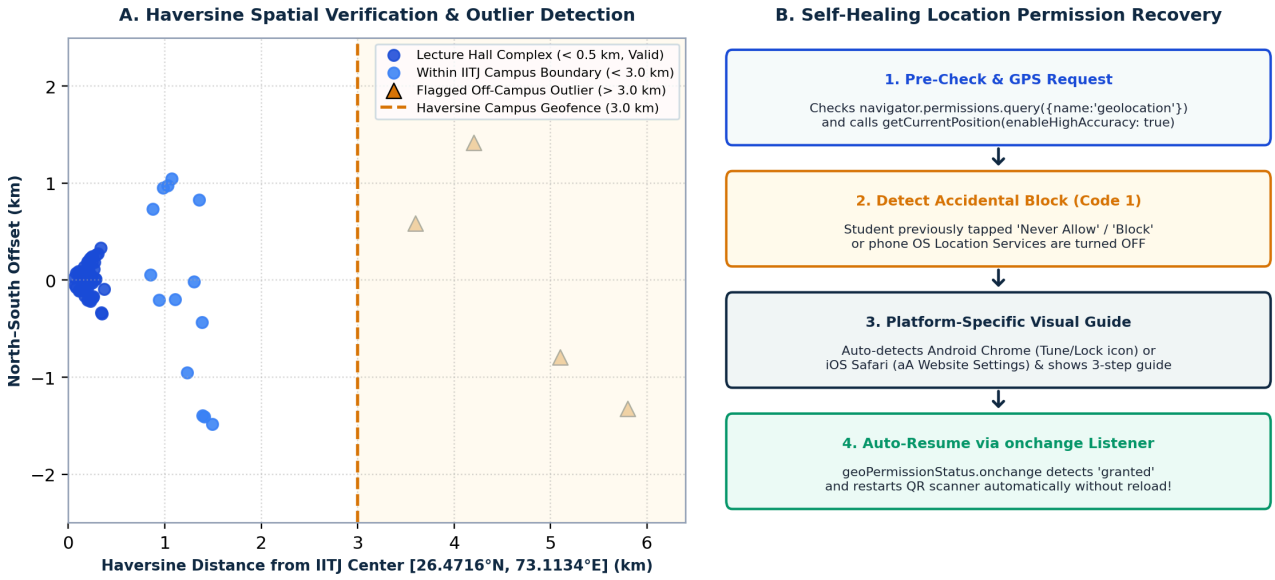


Figure 5: (A) Haversine spatial verification around IIT Jodhpur (26.4716° N, 73.1134° E) showing valid campus scans vs. flagged off-campus outliers beyond 3.0 km, and (B) the self-healing Location Permission Recovery flow for students.

5.2 Easy Location Permission Recovery When Students Accidentally Tap "Never Allow"

During classroom testing, we noticed that if a student accidentally tapped **Block** or **Never Allow** on the browser's location prompt (`error.code === 1`), mobile browsers refuse to show the native permission popup again when the student retries. As shown in Panel B of Figure 5, we solved this in `attendance.js` and `index.html` with an interactive **Location Permission Recovery Box** (`#scan-location-help`):

- **Pre-Scan Permission Check:** Before starting the camera, `startAttendance()` checks `navigator.permissions.query({ name: 'geolocation' })`. If the state is already `'denied'`, it immediately displays the unblock guide instead of making the student scan first.
- **Platform-Aware Step-by-Step Instructions:** `getDeviceLocationSteps()` detects whether the student is on **Android Chrome**, **iOS Safari**, or **Desktop**, and whether the issue is a blocked browser permission (`code === 1`) or phone GPS turned off (`code === 2/3`), showing 3 simple steps pointing directly to the address bar's **Tune / Lock** or **aA** settings icon.

- **Automatic Scanner Restart (`onchange` Listener):** While the guide is open, `watchLocationPermissionGrant()` listens to `geoPermissionStatus.onchange`. The moment the student flips Location to **Allow** in their browser address bar, the app detects `state === 'granted'` and automatically restarts the QR scanner without needing a page reload.

Location Scenario	What the Student Sees on Screen	How Recovery Completes
Android Chrome Blocked (`code === 1`)	Step-by-step guide: Tap **Tune / Lock icon** in address bar → **Permissions** → switch **Location** to **Allow**	`geoPermissionStatus.onchange` auto-detects permission grant and opens camera immediately
iOS Safari Blocked (`code === 1`)	Step-by-step guide: Tap **aA / Page Settings** in address bar → **Website Settings** → **Location** → **Allow**	Student taps **Test Location & Re-prompt** button (`#btn-recheck-location`) to verify GPS and scan
Phone GPS Turned Off (`code === 2/3`)	Prompts student to swipe down Quick Settings (Android) or open Location Services (iOS) to turn GPS ON	Tapping **Test Location & Re-prompt** acquires a fresh high-accuracy GPS fix and launches the scanner

6. Relational Database Schema and Serverless API Modules

All course, session, passkey, and attendance data is stored in six normalized PostgreSQL tables on Supabase (`schema.sql` and `migration.sql`).

6.1 Database Tables and Integrity Constraints

Foreign keys across the six tables use `ON DELETE CASCADE` so deleting a course cleanly removes its sessions, enrollments, and staff assignments, while composite `UNIQUE` constraints on `enrollments(student_id, course_id)` and `attendance_records(student_id, session_id)` prevent duplicate enrollments or double attendance entries at the database level (returning PostgreSQL error code `23505`). Table 6 summarizes each database table, and Figure 6 illustrates the Entity-Relationship (ER) schema.

Table Name	Key Columns	Role in the System
`students`	`id`, `email`, `roll_number`, `name`, `webauthn_credential`, `current_challenge`, `pending_qr_payload`, `challenge_expires_at`	Stores student profile, bound hardware passkey public key & counter, and active 30-second challenge state
`courses`	`id`, `course_code`, `course_name`, `instructor_email`, `created_at`	Stores courses created by authorized professors (`AUTHORIZED_PROFESSOR_EMAILS`)
`course_staff`	`id`, `course_id`, `email`, `role` (`'TA'`)	Maps Teaching Assistants (`@iitj.ac.in`) to courses so TAs can run live sessions and manage attendance
`enrollments`	`id`, `student_id`, `course_id`, `created_at`	Links students to the courses they join using the instructor's course code
`sessions`	`id`, `course_id`, `session_date`, `is_active`, `created_by`, `session_secret`	Represents each lecture session and stores the 256-bit secret key used to sign rotating QR codes

Table Name	Key Columns	Role in the System
`attendance_records`	`id`, `student_id`, `session_id`, `marked_at`, `latitude`, `longitude`, `marked_by`	Stores every verified student check-in along with live GPS coordinates and manual override attribution

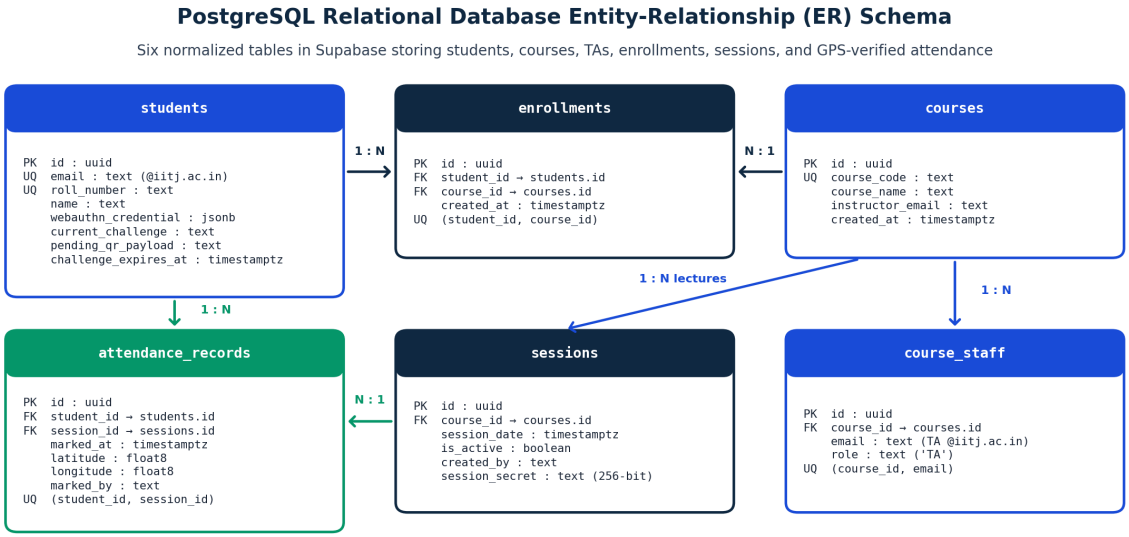


Figure 6: Entity-Relationship (ER) diagram of the six PostgreSQL tables (`students`, `enrollments`, `courses`, `attendance_records`, `sessions`, and `course_staff`) showing primary keys, unique constraints, and 1-to-many relationships.

6.2 Complete Backend API Module Reference (`lib/`)

All 18 backend modules in `lib/` are routed through `api/[...route].js`. Table 7 explains what each backend file does and how it validates requests.

Backend File (`lib/`)	Method	Who Can Call It	Function and Validation Logic
`auth-guard.js` & `config.js`	Helper	All API routes	Verifies Supabase JWT, checks `@iitj.ac.in` Google login, and defines timing constants & `qrBucketIsValid()`
`teacher-guard.js`	Helper	Teacher API routes	Checks if the caller is the course `INSTRUCTOR` or an assigned `TA` via `verifyCourseAccess()`
`student-data.js`	`GET / POST`	Authenticated Student	Creates student record on first login from email prefix; enrolls student in a course by `course_code`
`student-records.js`	`GET`	Authenticated Student	Returns total lectures, attended classes, attendance percentage, and date-sorted lecture history per course
`register-options.js` & `register-verify.js`	`POST`	Student (Unbound Device)	Issues and verifies the one-time FIDO2 platform passkey registration; blocks self-service device changes
`attendance-options.js`	`POST`	Enrolled Student	Verifies QR HMAC signature & 4s bucket grace window, checks course match, and issues 30s passkey challenge

Backend File (`lib/`)	Method	Who Can Call It	Function and Validation Logic
`mark-attendance.js`	`POST`	Enrolled Student	Atomically consumes challenge, verifies biometric signature & GPS coordinates, and records attendance
`edit-auth-options.js`, `edit-auth-verify.js` & `update-profile.js`	`POST / PATCH`	Authenticated Student	Requires biometric passkey verification to issue a 2-minute `edit_token` JWT before updating student name
`teacher-courses.js` & `manage-ta.js`	`GET / POST`	Instructor / TA	Lists courses, creates/deletes courses (professor), and adds or removes course TAs
`start-session.js`, `session-qr.js`, `session-attendance.js` & `end-session.js`	`POST / GET`	Course Instructor / TA	Starts/resumes live session, generates 4s HMAC QR payloads, polls live check-in count, and ends session
`teacher-records.js` & `course-locations.js`	`GET`	Course Instructor / TA	Returns student attendance roster, passkey binding status (`has_device`), and GPS points for the Leaflet map
`add-attendance-manual.js` & `remove-enrollment.js`	`POST`	Course Instructor / TA	Marks or unmarks attendance by roll number for any lecture and unenrolls students from a course
`teacher-reset-device.js` & `fix-roll-number.js`	`POST`	Instructor / TA (or Professor)	Unbinds a student's passkey when they get a new phone; allows professors to correct student roll numbers

7. Student Portal and Instructor Console Workflows

Both frontend interfaces are designed to make daily classroom use fast and effortless for students and instructors.

7.1 Student Portal Features (`index.html` & `attendance.js`)

- **75% Attendance Policy Calculator (`attendanceGuidance`):** For every enrolled course with A attended classes out of T total lectures, the app calculates exact guidance. If $A / T < 0.75$, it solves $(A + x) / (T + x) \geq 0.75 \Rightarrow x = \lceil 3T - 4A \rceil$ and tells the student: *"Attend next x classes for 75%"*. If $A / T \geq 0.75$, it solves $A / (T + y) \geq 0.75 \Rightarrow y = \lfloor (4A - 3T) / 3 \rfloor$ and shows: *"On track · y class buffer"*.
- **Two-Finger Camera Pinch-to-Zoom (`enablePinchToZoom`):** Students sitting far from the projector can pinch with two fingers on the camera screen to zoom in smoothly via `html5QrCode.applyVideoConstraints({ advanced: [{ zoom }] })`.
- **Course Detail & Lecture Filter:** Tapping any course opens a detailed lecture history with an animated SVG donut chart and segmented filters (`All`, `Present`, `Absent`).

7.2 Instructor Console Features (`teacher.html` & `teacher.js`)

- **Maximized Fullscreen Live QR Stage:** The **Live Session** tab renders a large `520 × 520` QR code (expanding up to `650 × 650` in Fullscreen mode) with an animated 4-second progress bar and a live counter of verified students. Confirmation modals (`confirmSheet`) and toasts mount dynamically inside `document.fullscreenElement || document.body` so clicking **End Session** works seamlessly even while in Fullscreen mode.

- **One-Click Interactive Attendance Matrix:** In the **Attendance Sheet** tab, instructors can click any ✓ or · cell in the student-by-date table to toggle attendance directly, filter by roll number, or use the manual entry bar with roll-number autocomplete.
- **Device Reset, Roll-Number Correction & Safe CSV Export:** Instructors and TAs can see whether each student's phone passkey is `Bound` or `No Device`, reset a student's bound device in one click, correct roll numbers, and export CSV sheets protected against spreadsheet formula injection.

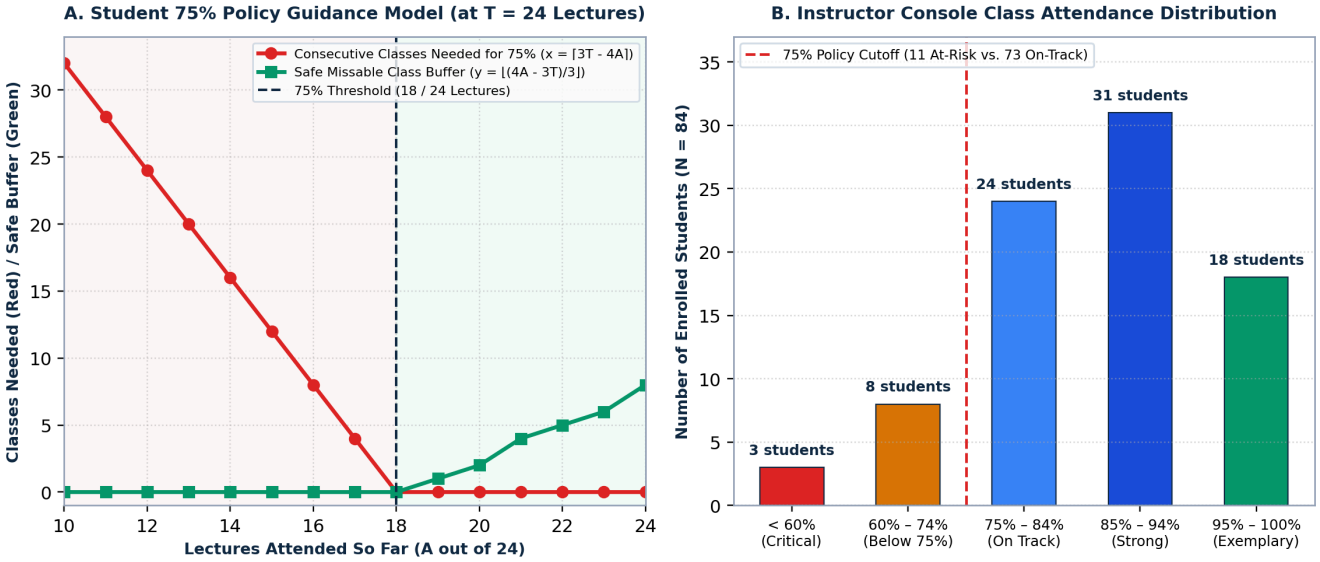


Figure 7: (A) The Student Portal's `attendanceGuidance` model showing exact classes needed to reach 75% vs. safe missable buffer classes across 24 held lectures, and (B) the Instructor Console's class attendance breakdown separating `<75%` at-risk students from on-track students.

Frontend Function	File	What It Does
`attendanceGuidance(a, t)`	`attendance.js`	Computes exact consecutive classes needed for 75% (*⌈3T − 4A⌉*) or safe missable buffer (*⌊(4A − 3T)/3⌋*)
`startAttendance(id, name)`	`attendance.js`	Pre-checks location permission, starts 12 FPS camera & 30s countdown, and runs passkey + GPS in parallel
`watchLocationPermissionGrant()`	`attendance.js`	Listens for `navigator.permissions` state change to `'granted'` and auto-restarts the QR scanner
`refreshQr()` & `animateQrProgressBar()`	`teacher.js`	Fetches new HMAC QR code every 4 seconds and animates the visual rotation bar
`renderGridTableOnly()`	`teacher.js`	Builds the interactive student-by-lecture matrix where clicking any cell toggles present/absent status
`drawSelectedSession()`	`teacher.js`	Plots lecture GPS check-ins on the Leaflet map and flags scans more than 3.0 km from IIT Jodhpur
`confirmSheet()` & `ensureToastStack()`	`shared.js`	Mounts confirmation dialogs and toasts inside `document.fullscreenElement || document.body`

8. Conclusions and Future Scope

The IIT Jodhpur Attendance Portal successfully achieves its core Semester 3 Design Credit objective: building a fair, fast, and proxy-resistant classroom attendance system without requiring dedicated hardware scanners in lecture halls. By combining institute Google OAuth 2.0 login, 4-second rotating HMAC-SHA256 QR codes, hardware-locked W3C WebAuthn biometric passkeys (with instructor-controlled device resets), and live Haversine GPS verification, the system blocks photo forwarding, multi-account phone sharing, and remote check-ins while allowing an entire class to mark attendance in under 30 seconds. Future enhancements can include optional ultrasonic audio tokens for indoor room-level verification and automated gradebook export to the IIT Jodhpur ERP portal.

9. Open-Source Libraries, Standards, and References

1. Indian Institute of Technology Jodhpur, "Dr. Pallavi Jain — Faculty Profile, Department of Computer Science and Engineering," https://iitj.ac.in/People/Profile/d91efa87-e52b-427e-9bde-1bd4c70da809?ln=en.
2. Mayank Tiwari (B25CM1075) and Shubham Bhatia (B25EC1050), "Attendance-portal: IIT Jodhpur Design Credit Project Repository," GitHub, https://github.com/MayankMT1024/Attendance-portal.
3. World Wide Web Consortium (W3C), "Web Authentication: An API for accessing Public Key Credentials — Level 3," W3C Recommendation, https://www.w3.org/TR/webauthn-3/.
4. Matthew Miller, "SimpleWebAuthn: TypeScript-First Libraries for Simpler WebAuthn Integration (@simplewebauthn/browser & @simplewebauthn/server v14)," https://simplewebauthn.dev/docs/.
5. H. Krawczyk, M. Bellare, and R. Canetti, "RFC 2104: HMAC: Keyed-Hashing for Message Authentication," Internet Engineering Task Force (IETF), https://www.rfc-editor.org/info/rfc2104/.
6. Supabase Inc., "Supabase Open-Source Backend, Authentication, and PostgreSQL Documentation," https://supabase.com/docs.
7. Minhazav (mebjas), "html5-qrcode: A Cross-Platform HTML5 QR Code Reader Library (v2.3.8)," GitHub, https://github.com/mebjas/html5-qrcode.
8. Sangmin Shim (davidshimjs), "QRCode.js: Cross-Browser QRCode Generator for JavaScript (v1.0.0)," GitHub, https://github.com/davidshimjs/qrcodejs.
9. Mozilla Developer Network (MDN), "Geolocation API and navigator.geolocation.getCurrentPosition() Specification," https://developer.mozilla.org/en-US/docs/Web/API/Geolocation_API.
10. Volodymyr Agafonkin, "Leaflet: An Open-Source JavaScript Library for Mobile-Friendly Interactive Maps (v1.9.4 Reference)," https://leafletjs.com/reference.html.